ANTHROPIC

Engineering Blog

中文精讀版

Sep 2024 — Mar 2026 | 12 篇精選文章 | 完整技術解析

anthropic.com/engineering

本書彙整 Anthropic Engineering Blog 自 2024 年 9 月至 2026 年 3 月的 12 篇核心技術文章，涵蓋 AI 代理架構、上下文工程、多代理系統、MCP 協議、評估方法等主題，並以繁體中文深度解析每篇文章的背景知識、技術細節與實作要點，適合希望深入了解前沿 AI 工程實踐的中文讀者。

代理架構 Agents	上下文工程 Context	多代理系統 Multi-Agent
RAG 與檢索	MCP 與工具	評估方法 Evals

關於本書

Anthropic 是 Claude 背後的 AI 安全公司，其工程部落格（Engineering Blog）是業界少數能夠真正揭示前沿 AI 系統設計思路的技術資源。這些文章不只分享理論，更詳細記錄了 Anthropic 內部團隊在實際建構 Claude、Claude Code、Research 功能等產品時所踩過的坑、學到的教訓，以及最終選擇的架構決策。

本書的目標讀者是有一定技術背景、希望了解 AI 代理（AI Agents）工程實踐的中文讀者。書中每篇文章均包含以下幾個部分：

- 背景知識 — 解釋理解本文所需的前置概念
- 問題定義 — 說明 Anthropic 試圖解決的核心問題
- 技術解析 — 深入分析文章提出的方法與架構
- 核心數據 — 列出文章中的重要量化結果
- 實作要點 — 整理可直接應用的工程建議
- 延伸思考 — 探討技術的局限性與未來方向

建議閱讀順序：如果你是 AI 工程新手，建議從 01（建構有效代理）和 02（Contextual Retrieval）開始；如果你已有一定基礎，可以直接跳到 06（多代理研究系統）或 08（上下文工程）等進階主題。

目錄 Contents

NO. 01 · Dec 2024

Building Effective Agents

代理架構的基礎設計原則

NO. 02 · Sep 2024

Introducing Contextual Retrieval

突破 RAG 準確率瓶頸的關鍵技術

NO. 03 · Jan 2025

Raising the Bar on SWE-bench Verified

AI 如何在真實軟體工程任務中超越人類基準

NO. 04 · Mar 2025

The 'Think' Tool

讓 Claude 在行動前暫停深思的設計

NO. 05 · Apr 2025

Claude Code: Best Practices

代理式程式設計的工程最佳實踐

NO. 06 · Jun 2025

Multi-Agent Research System

多代理並行架構如何實現深度研究

NO. 07 · Jun 2025

Desktop Extensions & MCP

一鍵安裝 MCP 伺服器的生態建設

NO. 08 · Sep 2025

Effective Context Engineering

從提示工程進化到上下文工程

NO. 09 · Nov 2025

Code Execution with MCP

用程式碼降低 98.7% 的 token 消耗

NO. 10 · Nov 2025

Effective Harnesses for Long-Running Agents

跨越多個上下文窗口的長時任務框架

NO. 11 · Jan 2026

Demystifying Evals for AI Agents

AI 代理評估的體系與方法論

NO. 12 · Feb 2026

Building a C Compiler with Parallel Claudes

16 個並行 Claude 協作構建編譯器

01

AGENTS · ARCHITECTURE · WORKFLOWS

Building Effective Agents

December 19, 2024 · anthropic.com/engineering/building-effective-agents

背景知識

大型語言模型（LLM）最初的使用方式主要是單次問答：給一個提示，得到一個回應。隨著模型能力的提升，開發者開始讓 LLM 連接工具（Tools），讓它能夠搜尋網路、執行程式碼、讀寫資料庫——這就是「工具使用（Tool Use）」的概念。當 LLM 在一個迴圈中不斷呼叫工具、根據結果調整下一步行動，就形成了所謂的「代理（Agent）」。

2024 年 AI 代理的討論極度熱門，但也極度混亂：每個廠商、每個開源框架都有自己對「代理」的定義，導致開發者難以判斷什麼架構適合自己的需求。Anthropic
這篇文章的價值在於提供了一個清晰的分類框架，幫助開發者從第一原則（first principles）思考系統設計。

問題定義

Anthropic 工程師 Erik Schluntz 和 Barry Zhang 在與數十個客戶團隊合作後觀察到一個現象：最成功的 AI 代理實作，往往不是最複雜的那些。很多團隊過早引入複雜的框架（如 LangChain、AutoGen 等），反而讓系統變得脆弱、難以調試。

核心問題是：開發者不知道什麼時候應該用「工作流（Workflow）」，什麼時候應該用「代理（Agent）」，也不知道如何設計工具和提示詞才能讓系統可靠運作。

核心架構分類

文章提出了一個重要的架構區分，把所有 AI 自動化系統統稱為「代理式系統（Agentic Systems）」，再細分為兩類：

工作流 Workflows	LLM 和工具透過預先定義的程式碼路徑來協調。每個步驟都是確定的， 就像一個流程圖——從 A 到 B 再到 C，不會偏離。 適合：可預測的重複任務，需要高穩定性的場景
代理 Agents	LLM 動態決定自己的流程和工具使用方式，維持對任務完成方式的控制權。 代理根據環境回饋（如工具呼叫的結果）來決定下一步。 適合：開放式問題、難以預測步驟數量的複雜任務

五大工作流模式

文章詳細介紹了五種常見的工作流模式，每種都有其適用場景：

① 提示鏈（Prompt Chaining）

最簡單的模式：把一個複雜任務分解成一系列子任務，前一個步驟的輸出成為下一個步驟的輸入。就像工廠的生產線，每個工站只做一件事。例如：先生成文章大綱，再根據大綱寫每個章節，最後檢查格式。適合任何可以線性分解的任務，特別是當每個步驟都需要不同的「角色設定」時。

② 路由（Routing）

讓 LLM 先判斷輸入屬於哪種類型，再分配給對應的專門處理流程。例如客服系統：先判斷是退款問題、技術問題還是一般諮詢，再路由到對應的處理流程。這樣可以針對不同類型的問題優化提示詞，避免一個萬用提示詞什麼都做得一般般。

③ 並行化（Parallelization）

讓多個 LLM 同時處理同一任務，有兩種變體：「分段（Sectioning）」是把大任務拆成獨立的小任務並行執行；「投票（Voting）」是對同一問題跑多次，取最多票的答案。投票適合需要高可靠性的場景，例如程式碼安全審查——多個不同提示詞的模型各自找漏洞，只要有一個發現就標記。

④ 協調者—工作者（Orchestrator-Workers）

一個主要的 LLM（協調者）負責分解任務、分配給多個子 LLM（工作者），最後彙整結果。這是建構多代理系統的基礎模式。適合複雜任務，例如需要同時修改多個程式碼檔案的工程任務。

⑤ 評估者—優化者（Evaluator-Optimizer）

一個 LLM 負責生成，另一個負責評估並給出改善回饋，形成迭代迴圈直到滿足品質標準。適合有明確評估標準且迭代優化有價值的任務，如文學翻譯、需要多輪搜尋的複雜研究任務。

核心設計原則

文章提出三大原則，Anthropic 稱之為構建代理的「哲學」：

- 保持設計簡單（Maintain simplicity）：抵抗過度工程化的誘惑，只在能明確證明複雜度帶來收益時才增加複雜度
- 透明規劃步驟（Prioritize transparency）：讓代理的規劃步驟可見，方便調試和理解
- 謹慎建構（Build carefully）：充分測試和評估後再部署，尤其是有副作用的操作

工具設計的重要性

文章特別強調工具設計（Tool Design）往往被低估。工具定義和說明文件應該和整體提示詞一樣受到重視。好的工具設計原則包括：

- 給工具清楚的命名和詳細的說明，讓模型明確知道何時應該用這個工具
- 同一個操作可能有多種表達方式（如「寫差異（diff）」vs「重寫整個檔案」），選擇最適合當前情境的格式
- 工具的輸出格式要讓模型容易解讀，避免傳回冗長的原始資料

什麼時候不需要代理

文章特別指出，很多情況下根本不需要代理系統。如果你的知識庫小於 20 萬個 token（約 500 頁），直接把整個知識庫塞進 Claude 的上下文可能比建 RAG 系統更簡單有效。代理系統通常以延遲（latency）和成本換取更好的任務表現，這個取捨不是在所有情境下都合理。

70+	5	3	200K
合作客戶團隊數	工作流模式	代理設計原則	直接塞入 context 的 token 上限

延伸思考

這篇文章最重要的貢獻不是任何具體技術，而是思維框架的澄清。在 AI 代理的浪潮中，很多開發者的直覺是「越智能越好」、「越自主越好」，但 Anthropic 的觀察恰恰相反——對於大多數商業場景，可預測性和可調試性往往比靈活性更有價值。框架（Framework）的吸引力在於快速上手，但代價是失去透明度和控制權，這對於生產環境中需要穩定運行的系統是一個嚴重的隱患。

02

RAG · RETRIEVAL · EMBEDDINGS · BM25

Introducing Contextual Retrieval

September 19, 2024 · anthropic.com/engineering/contextual-retrieval

背景知識：什麼是 RAG?

RAG (Retrieval-Augmented Generation，檢索增強生成) 是讓 AI 模型「查資料再回答」的技術。LLM 的訓練資料有截止日期，也無法包含企業內部的私有資料。RAG 的解法是：建立一個知識庫，使用者提問時先從知識庫中找出最相關的資料片段，再把這些片段和問題一起送給 LLM，讓它基於這些資料來回答。

傳統 RAG 的運作流程：(1) 把文件切割成小塊 (chunks)，通常每塊幾百個 token；(2) 用嵌入模型 (embedding model) 把每個 chunk 轉成向量；(3) 把所有向量存入向量資料庫；(4) 使用者查詢時，把問題也轉成向量，找出最相近的幾個 chunk；(5) 把這些 chunk 加入提示詞，讓 LLM 生成回答。

問題所在：切割後的失憶

傳統 RAG 有一個根本性的缺陷：當文件被切割成小塊後，每個 chunk 可能失去了理解它所需的上下文。Anthropic 給了一個生動的例子——假設你有一份公司的財務報告，其中有一個 chunk 寫著：

> 「該公司的營收比上一季增加了 3%。」

這個 chunk 本身完全沒有問題，但是它沒有說明這是哪家公司、哪個季度的數據。當向量搜尋找到這個 chunk 時，模型根本無法確定它是否和使用者的問題相關。這就是「脈絡遺失 (Context Loss)」問題。類似的問題在技術文件中更嚴重：一個描述錯誤代碼「TS-999」的 chunk，向量搜尋可能找不到它，因為語意搜尋擅長的是「概念相似」，不擅長「精確字串匹配」。

Contextual Retrieval 解決方案

Anthropic 提出的解法非常直觀：在把每個 chunk 放入向量資料庫之前，先用 Claude 為它加上一段簡短的上下文說明 (contextual prefix)，讓 chunk 本身就包含理解它所需的關鍵資訊。

原始 chunk	「該公司的營收比上一季增加了 3%。」
加入上下文後	「本段摘自 ACME 公司 2023 年 Q2 財報，上季營收為 3.14 億美元。該公司的營收比上一季增加了 3%。」

這個做法在技術上非常簡單，但效果卻相當顯著。加入上下文的 chunk 在嵌入時能夠捕捉到更豐富的語意，BM25 (精確字串匹配算法) 也能更準確地匹配到相關內容。

技術細節：Claude 如何生成上下文

Anthropic 使用的提示詞模板如下（簡化版）：

> 系統：給你一份完整文件和其中的一個段落，請用 1-2 句話描述這個段落在文件中的位置和作用，僅回覆這段簡短說明，不要加任何前言。

這個任務本身很簡單，Claude 生成的上下文通常只有 50-100 個 token，成本很低。更重要的是，Anthropic 利用了「提示詞快取（Prompt Caching）」功能——整份文件只需要載入一次，後續為每個 chunk 生成上下文時都可以複用已快取的文件，大幅降低成本。根據 Anthropic 的計算，以每百萬文件 token 計算，生成外有 chunk 上下文的一次性費用約為 1.02 美元。

混合搜尋：嵌入向量 + BM25

Contextual Retrieval 還結合了兩種不同的搜尋方式：

- 語意嵌入搜尋（Semantic Embedding Search）：擅長找概念相似的內容，即使用詞不同
- BM25 關鍵字搜尋：擅長精確匹配，例如產品名稱、錯誤代碼、人名

兩種方式各自找出最相關的 chunks，再用排名融合（Rank Fusion）合併去重，取最相關的 K 個塊送給 LLM。若再加上重新排序（Reranking），由另一個模型對候選結果進行精排，效果還能再提升。

49%	67%	$1.02	200K
降低檢索失敗率（單用上下文嵌入+BM25）	降低檢索失敗率（加入 Reranking）	每百萬文件 token 的上下文生成成本	低於此 token 數建議直接塞入 context

實作要點

- 嵌入模型推薦 Voyage AI 或 Gemini；Reranking 模型推薦 Cohere
- 實驗顯示取前 20 個 chunks 效果最好（而非常見的 top-5）
- 可以為不同領域撰寫客製化的上下文生成提示詞，效果通常優於通用提示詞
- 知識庫 <500 頁（約 20 萬 token）時，考慮直接把整個知識庫塞進 context

延伸思考

Contextual Retrieval 展示了一個重要的工程思路：用 LLM 的生成能力來改善 LLM 的檢索能力。許多 RAG 系統的瓶頸不在生成，而在檢索——找不到對的資料，再強的生成模型也無濟於事。這個技術對任何需要知識庫的 AI 應用都有參考價值。

03

BENCHMARKS · CODING AGENTS · EVALUATION

Raising the Bar on SWE-bench Verified with Claude 3.5 Sonnet

January 6, 2025 · anthropic.com/engineering/swe-bench-sonnet

背景知識：SWE-bench 是什麼？

SWE-bench（Software Engineering Benchmark）是一個評估 AI 軟體工程能力的基準測試。它從真實的 GitHub 開源專案中收集 Pull Request（PR），要求 AI 代理僅根據 PR 描述（不看代碼變更）來修復對應的 issue。這個測試的難度在於它是「真實世界」的問題：不是精心設計的教科書題目，而是真實開發者遇到的 bug 和功能需求，需要理解整個程式碼庫的架構才能解決。

SWE-bench Verified 是其精選版本，由人工審核確認問題描述夠清楚、測試夠完整，以提高評估的可信度，避免因問題本身不明確而造成的假陰性（false negative）。

AI 在程式碼任務的優勢

程式碼是 AI 代理最適合發揮的領域之一，原因很簡單：程式碼有自動化的驗證機制——測試。和其他任務（如寫文章、做分析）不同，程式碼有「對或錯」的客觀標準，代理可以執行測試、根據測試結果調整代碼，形成一個可靠的反饋迴圈。

Claude 3.5 Sonnet 的代理架構

為了解決 SWE-bench 問題，Claude 3.5 Sonnet 被部署在一個代理迴圈中，配備了以下工具：

- 程式碼庫搜尋：在整個 repo 中搜尋相關函式、類別、檔案
- 檔案讀寫：讀取現有程式碼、修改相關部分
- 命令執行：執行測試、安裝依賴、驗證修復
- 錯誤分析：解讀測試失敗訊息，決定下一步修改方向

代理的工作流程類似於人類工程師：閱讀 issue 描述 → 定位相關代碼 → 理解問題根因 → 提出修復方案 → 執行測試 → 根據測試結果反覆迭代直到通過。

為什麼這很重要

SWE-bench 的成績提升不只是一個數字——它反映了 AI 在實際軟體工程工作中的能力邊界。當 AI 能夠可靠地解決真實的 GitHub issues，它就開始從「程式碼補全工具」進化為「自主軟體工程師」。這對整個軟體開發行業的工作方式有深遠影響。

然而文章也誠實地指出人工審查的必要性：即使代理解決了問題，也需要人類確認解決方案符合系統的整體設計哲學和長期維護性，而不只是讓測試通過的「補丁式」修復。

延伸思考

SWE-bench 的侷限性在於它主要測試「修 bug」的能力，而真實的軟體工程工作還包括架構設計、需求分析、代碼審查、技術債管理等難以量化的面向。未來的評估基準需要涵蓋更廣泛的工程技能，才能真正衡量 AI 作為工程師的全面能力。

04

TOOL USE · REASONING · COMPLEX DECISIONS

The 'Think' Tool: Enabling Claude to Stop and Think in Complex Tool Use Situations

March 20, 2025 · anthropic.com/engineering/claude-think-tool

背景知識：工具使用的挑戰

當 Claude 在代理迴圈中使用工具時，它需要在「呼叫工具」和「生成回應」之間不斷切換。在簡單任務中這沒問題，但遇到複雜場景——例如需要遵循複雜的業務規則、追蹤多個相互依賴的狀態、或在多個可能的操作路徑中選擇——模型往往會跳過必要的分析步驟，直接採取行動，導致錯誤。

「擴展思考（Extended Thinking）」是 Claude 的一個功能，讓模型在生成最終回應前輸出一串可見的思考過程。但 Extended Thinking
主要設計用於「一次性的複雜推理任務」，不太適合代理場景中那種「每次工具呼叫後都需要短暫評估」的需求。

「think」工具的設計

解法出乎意料地簡單：在工具清單中加入一個名為「think」的工具，讓 Claude
在需要的時候主動呼叫它。這個工具不執行任何外部操作，只是給 Claude 一個結構化的空間來整理思路。

工具名稱	think
輸入參數	thought（字串）：Claude 想要記錄的思考內容
返回值	空字串（工具本身不產生任何資訊，只是讓思考過程被記錄）
作用	在行動前進行結構化思考，不消耗工具呼叫的「配額」

適用場景

根據 Anthropic 的測試，「think」工具在以下場景特別有效：

- 複雜政策解讀：需要對照業務規則決定是否批准某個操作，且規則之間有依賴關係
- 多步驟工具序列：工具 A 的結果影響是否需要工具 B，工具 B 的結果又影響 C 的參數選擇
- 代碼除錯：在閱讀錯誤訊息後，需要分析可能的根因再決定修改哪個部分
- 需要遵守約束條件：行動前需要確認不違反某些限制（如不能刪除某些資料）

與 Extended Thinking 的差異

think 工具	在代理迴圈中按需呼叫，適合每次工具結果後的短暫評估
Extended Thinking	用於生成最終回應前的深度推理，適合需要長時間思考的單一問題
interleaved thinking	Extended Thinking 的一種模式，在工具呼叫之間自動插入思考

實作要點

「think」工具的實作成本極低，只需在工具定義中加入幾行：

```
{ "name": "think", "description": "Use this tool to think through a problem before acting. Helps with complex decisions and multi-step reasoning.", "input_schema": { "type": "object", "properties": { "thought": { "type": "string" } }, "required": ["thought"] } }
```

關鍵是在系統提示詞中說明什麼時候應該使用「think」工具，避免模型對所有問題都呼叫它（浪費 token）或完全不使用（失去效果）。

延伸思考

「think」工具體現了一個有趣的設計哲學：與其試圖讓模型「自動地」在需要時思考得更深，不如給它一個明確的機制，讓它「知道自己可以暫停和思考」。這也提醒我們，很多 AI 能力的提升不來自更大的模型，而來自更好的工具設計和提示工程。

05

CLAUDE CODE · AGENTIC CODING · BEST PRACTICES

Claude Code: Best Practices for Agentic Coding

April 18, 2025 · anthropic.com/engineering/claude-code-best-practices

背景知識：什麼是 Claude Code?

Claude Code 是 Anthropic 推出的命令列程式設計工具，讓 Claude 能夠直接存取你的程式碼庫、編輯檔案、執行命令、搜尋文件。它不只是程式碼補全工具，而是能夠自主完成完整工程任務的代理。Claude Code 最初是為了提升 Anthropic 內部開發效率而建立的，後來開放給外部開發者使用。

給 Claude Code 的四大建議

1. 給予充分的任務說明和上下文

Claude Code 的效果很大程度上取決於你給它的指令品質。模糊的高層次指令（如「修好這個 bug」）往往效果不佳，因為 Claude 需要先花大量時間探索代碼庫來理解問題。更好的做法是提供：問題的精確描述、相關的錯誤訊息、可能涉及的檔案或模組、你期望的行為和實際行為的差異。

2. 使用 CLAUDE.md 管理持久上下文

CLAUDE.md 是一個特殊的 Markdown 檔案，放在專案根目錄或子目錄中，Claude Code 每次啟動時都會自動讀取它。你可以用它記錄：

- 專案架構說明（目錄結構、主要模組的職責）
- 編碼慣例（命名規範、格式規範、偏好的設計模式）
- 常用命令（如何跑測試、如何啟動開發伺服器）
- 已知的限制和陷阱（不應修改的檔案、特殊的業務邏輯）

3. 把大任務分解成小任務

雖然 Claude Code 能夠處理複雜的多步驟任務，但把大任務分解成更小的可驗證單元，往往能得到更好的結果。每個小任務完成後都可以確認結果正確，避免在錯誤的方向上走太遠才發現問題。

4. 善用 Git 作為安全網

鼓勵 Claude Code 在每個有意義的中間狀態都做一次 git commit。這樣即使後續操作出錯，也可以輕易回退到上一個正常狀態。同時，有意義的 commit 訊息也有助於追蹤代理的工作進展，這對長時任務（後面的文章會詳細討論）尤為重要。

Claude Code 的工具集

Claude Code 和人類開發者使用相同的工具集，這是它能夠有效工作的關鍵：

- bash/shell：執行任意系統命令，包括跑測試、安裝依賴、git 操作
- 文字編輯器操作：讀取、修改、建立、刪除檔案
- 搜尋工具：grep、find、ripgrep 在整個程式碼庫中搜尋
- 瀏覽器（選用）：測試 Web UI 的視覺效果

延伸思考

Claude Code 的最佳實踐在某種程度上也是一種對「代理能力邊界」的誠實告白。模型再強大，如果沒有清楚的任務定義、沒有持久的上下文管理、沒有可靠的狀態保存機制，在實際工程任務中的表現依然有限。「代理 = 模型 + 環境設計」，後者往往決定了上限。

06

MULTI-AGENT · RESEARCH · PARALLEL ARCHITECTURE

How We Built Our Multi-Agent Research System

June 13, 2025 · anthropic.com/engineering/multi-agent-research-system

背景知識：為什麼需要多代理？

單一代理的研究能力受限於上下文窗口大小和循序處理的速度。當一個研究問題需要同時探索 10 個不同的方向時，單代理只能一個接一個地搜尋，不只耗時，還容易因為上下文積累太多無關資訊而「迷失」。多代理系統的核心優勢是：並行探索 + 乾淨的上下文隔離。

系統架構：LeadResearcher + Subagents

Anthropic 的 Research 功能採用了明確的層級架構：

主代理（LeadResearcher）：負責解析使用者的研究問題，制定研究計畫，將問題分解為多個子任務，並最終整合所有子代理的發現，生成有引用來源的完整報告。

子代理（Subagents）：每個子代理被分配一個具體的研究子任務（如「找出 X 公司的董事會成員」），獨立執行搜尋、評估資訊品質、整理發現，最後以簡潔的摘要（1,000–2,000 token）回報給主代理。子代理擁有完全獨立的上下文，不受其他子代理的干擾。

並行化的設計細節

Anthropic 實作了兩個層次的並行：

- 層次一：主代理同時啟動 3–5 個子代理並行探索不同方向，而非依序
- 層次二：每個子代理在執行時也同時發出多個搜尋工具呼叫（而非一次一個）

兩個層次的並行組合，讓複雜研究任務的完成速度提升數倍。Anthropic 發現早期的循序搜尋版本「慢到令人痛苦」，這是推動並行化設計的直接動力。

Extended Thinking 的角色

主代理使用 Extended Thinking（擴展思考模式）來規劃研究策略——評估任務複雜度、決定需要幾個子代理、為每個子代理定義具體任務。子代理在收到每次工具搜尋的結果後，也使用 interleaved thinking（交錯思考）來評估結果品質、判斷是否需要進一步搜尋。

關鍵數據與驗證

90.2%	3–5	3+	40%
多代理 vs 單代理的任務完成率提升	主代理啟動的子代理數量	每個子代理的並行工具呼叫數量	AI 自動改善工具描述後的任務時間減少

讓代理自我改善工具描述

這是文章中一個特別有趣的細節：Anthropic 建立了一個「工具測試代理（tool-testing agent）」，讓它反覆測試某個有缺陷的 MCP 工具，然後自動重寫工具說明來避免已知的陷阱。這個過程發現了工具使用的關鍵細節和 bug，最終使使用這個工具的代理任務時間減少了 40%。這是「用 AI 改善 AI 工具」的典型案例。

搜尋策略：先廣後深

文章指出代理常犯的一個錯誤：使用過於具體的長查詢，導致結果稀少。Anthropic 的策略是讓子代理先用短而廣泛的查詢探索，根據初步結果逐步縮小範圍。這模仿的是人類研究員的工作方式：先摸清「這個領域有什麼」，再深入挖掘具體問題。

評估方法

多代理系統的評估比單代理更複雜，因為中間過程（子代理的搜尋路徑、評估決策）也需要被監控。Anthropic 的評估使用 LLM 作為評判者（LLM-as-judge），設計結構化的評分量表衡量準確性、完整性、來源品質和引用正確性。人工測試員發現了一個自動評估遺漏的問題：早期代理偏好 SEO 優化的內容農場，而不是更權威的學術 PDF 或個人部落格。這促使 Anthropic 在提示詞中加入來源品質啟發式規則。

延伸思考

多代理系統的代價是 token 消耗量急劇上升——每個子代理各自的搜尋和處理，加上主代理的協調和整合，總 token 數可能是單代理的數倍到十倍。這意味著多代理架構適合「高價值的深度研究任務」，而非日常的簡單查詢。如何根據任務複雜度動態決定是否啟用多代理，是未來工程設計的重要課題。

07

CONTEXT ENGINEERING · AGENT ARCHITECTURE · MEMORY

Effective Context Engineering for AI Agents

September 29, 2025 · anthropic.com/engineering/effective-context-engineering-for-ai-agents

從提示工程到上下文工程

「提示工程（Prompt Engineering）」指的是如何撰寫好的提示詞，使 LLM 產出預期的回應。這在 LLM 初期是核心技能。但隨著代理系統越來越複雜、任務越來越長，提示詞只是整個「上下文（Context）」的一部分。上下文包含：系統提示、工具定義、MCP 連接、外部資料、對話歷史、工具呼叫結果等等。

「上下文工程（Context Engineering）」是更廣義的技術：在每次 LLM 推理時，精心策劃哪些 token 應該出現在上下文中，以最大化達成目標行為的可能性。核心挑戰是：上下文窗口是有限的資源，每個 token 都在競爭模型的注意力。

上下文的組成元素

系統提示詞	定義代理的角色、能力和限制。應簡潔且在「正確高度」—— 太高層次（缺乏具體指引）或太低層次（過度硬編碼邏輯）都是問題
工具定義	告訴模型可以使用哪些工具及其用途。 工具說明佔用大量 token，需要謹慎設計
MCP 資源	外部服務的連接資訊。工具數量過多時需要動態載入
外部資料	RAG 檢索結果、使用者上傳的文件等。需要精確過濾相關性
對話歷史	過去的交互記錄。隨著任務進行會急速增長，需要主動管理
工具結果	工具呼叫的返回值。可能包含大量冗餘資訊，需要在傳入前過濾

三大上下文管理策略

Anthropic 介紹了三種主要策略，分別適合不同的應用場景：

策略一：上下文壓縮（Compaction）

當對話歷史過長時，不是截斷它（會丟失重要資訊），而是用 LLM 對歷史記錄進行摘要。Claude Agent SDK 有內建的自動壓縮功能，當上下文接近窗口上限時觸發。適合需要長時間來回互動的任務，如客服對話、多輪研究。缺點是壓縮本身會消耗 token，且摘要可能遺失細節。

策略二：筆記機制（Note-taking）

讓代理在執行過程中主動記筆記，把重要的中間狀態、已完成的工作、待解決的問題記錄到外部存儲（如檔案或資料庫）。當上下文窗口用盡而需要開啟新的對話時，把筆記帶入新的上下文，實現跨窗口的連續工作。這正是 Anthropic 為長時任務設計的核心機制（後面的 Article 10 會詳述）。

策略三：多代理架構（Multi-Agent）

子代理各自擁有獨立、乾淨的上下文，只處理主代理分配的子任務，完成後只回傳精煉的摘要（而非整個探索過程）。這讓主代理的上下文保持整潔，不會被子任務的細節淹沒。同時，子代理可以並行運行，大幅提升效率。

系統提示詞的「正確高度」

文章特別強調系統提示詞需要在「Goldilocks Zone」（剛剛好的區間）：

- 太高層次：「你是一個有幫助的助手」——缺乏具體指引，代理不知道如何應對特定情境
- 太低層次：在提示詞中硬編碼複雜的業務邏輯——維護困難，容易出錯，且讓 LLM 難以推理
- 剛剛好：清晰的角色定義、明確的能力範圍、具體的行為指引、關鍵的限制條件

延伸思考

上下文工程的概念提示我們：在 AI 代理系統中，工程師的核心工作不是寫程式碼，而是設計資訊流——決定什麼資訊在什麼時候出現在模型的視野中。這是一種全新的工程思維方式，需要同時理解 LLM 的注意力機制和業務邏輯的資訊結構。

08

MCP · CODE EXECUTION · TOKEN EFFICIENCY

Code Execution with MCP: Building More Efficient Agents

November 4, 2025 · anthropic.com/engineering/code-execution-with-mcp

背景知識：MCP 是什麼？

MCP（Model Context Protocol，模型上下文協議）是 Anthropic 於 2024 年 11 月推出的開放標準，讓 AI 代理能夠連接各種外部服務和工具。在 MCP 之前，每個工具整合都需要客製化開發，造成大量重複工作。MCP 提供了一個通用協議——開發者只需實作一次，就能接入整個 MCP 生態系統。

MCP 的快速普及帶來了新問題：現在一個代理可能連接了數十甚至數百個 MCP 伺服器，擁有數千個可用工具。每次代理啟動時，都需要把這些工具的定義（名稱、說明、參數 schema）全部載入到上下文中——這可能消耗數十萬個 token，在代理開始處理任何任務之前就已付出巨大成本。

問題的具體量化

假設你有一個連接了 Salesforce 和 Google Drive 等服務的代理，需要完成以下任務：

> 「把我的 Google Drive 中的會議記錄下載下來，附到 Salesforce 的對應客戶記錄上。」

傳統的直接 MCP 工具呼叫方式：

① 載入所有 MCP 工具定義（大量 token）

② TOOL CALL: gdrive.getDocument() → 返回完整會議記錄文字（進入上下文）

③ TOOL CALL: salesforce.updateRecord() → 模型必須把整份記錄再次寫入參數（二次佔用上下文）

在這個例子中，完整的會議記錄在上下文中出現了兩次：一次作為工具呼叫的結果，一次作為 Salesforce 更新請求的參數。每個中間結果都必須流過模型，無論資料多大。

解決方案：用程式碼取代直接工具呼叫

核心想法是：讓代理寫程式碼來處理工具互動，而不是直接呼叫工具。代理可以寫一段腳本，在腳本中：動態載入需要的工具、在本地處理和過濾資料、只把必要的結果傳回給模型。整個中間過程在程式碼的執行環境中進行，不需要流過模型上下文。

動態工具探索

取代一次性載入所有工具定義，可以在每個 MCP 伺服器中加入一個 search_tools 工具，讓代理按需搜尋和載入工具。search_tools 支援詳細程度參數：

- 僅名稱（name only）：快速瀏覽有哪些工具，消耗最少 token
- 名稱和描述（name + description）：了解工具用途再決定是否需要

- 完整定義（full schema）：確定要使用時才載入完整的參數定義

150,000 傳統方式消耗的 token 數（所有工具定義）	2,000 程式碼執行方式消耗的 token 數	98.7% token 節省比例	2024/11 MCP 推出月份

延伸思考

這篇文章揭示了一個普遍的工程原則：LLM 上下文是稀缺資源，任何可以在「程式碼執行層」解決的問題，都不應該在「LLM 推理層」解決。隨著 AI 代理連接的外部系統越來越多，這種「代理操作電腦而非直接操作工具」的思路將越來越重要。

09

LONG-RUNNING AGENTS · HARNESS · MULTI-SESSION

Effective Harnesses for Long-Running Agents

November 26, 2025 · anthropic.com/engineering/effective-harnesses-for-long-running-agents

背景知識：為什麼長時任務很難？

構建一個能完成「寫一個完整 Web 應用」這類複雜任務的代理，面臨一個根本性的挑戰：上下文窗口是有限的。即使 Claude 的上下文窗口有 20 萬個 token，對於一個需要探索龐大程式碼庫、反覆迭代、執行大量測試的工程任務來說，依然遠遠不夠。

Claude Agent SDK 有「上下文壓縮（Compaction）」功能，理論上讓代理可以無限期地工作。但 Anthropic 發現，即使有了 Compaction，給 Opus 4.5 一個高層次的指令（如「克隆 claude.ai」），依然無法可靠地完成生產品質的 Web 應用。

兩大失敗模式

Anthropic 觀察到代理在長時任務中的兩種典型失敗：

失敗模式一：一次性嘗試（One-shotting）

代理試圖一次性完成所有事情，在上下文耗盡前盡可能多寫代碼，結果在上下文到達限制時留下一堆半成品功能——前端有但後端沒有、API 定義了但邏輯未實作、測試寫了但根本跑不過。這些半成品比沒有還難處理，因為下一個代理實倒看到的是一個「看起來快完成」但實際上四處都是坑的代碼庫。

失敗模式二：自我評估過於樂觀（Self-evaluation Bias）

代理傾向於對自己的工作給予過高評價。即使代碼有明顯問題，代理也常常認為已經完成了任務。「讓代理評估自己的工作」幾乎不可靠——就像讓一個作者校對自己的文章，很難發現明顯的錯誤。

解決方案：雙代理架構

Anthropic 設計了一個兩階段的代理框架：

階段一：初始化代理（Initializer Agent）

只在第一個上下文窗口運行，負責建立整個任務的基礎設施：

- 展開使用者的高層次需求，生成包含數百個具體可測試功能點的 JSON 需求列表
- 建立 Git 倉庫並做初始提交
- 建立 init.sh（啟動開發伺服器和測試的腳本）
- 建立 claude-progress.txt（記錄完成進度的追蹤文件）

階段二：編碼代理（Coding Agent）

在每個後續的上下文窗口中運行，每次都從 claude-progress.txt 和 Git 歷史中快速了解當前狀態，然後：

- 每次只完成幾個完整的功能（而非試圖一次完成全部）
- 使用 Puppeteer 等瀏覽器自動化工具驗證功能是否正確運作
- 只有在測試通過後才更新 claude-progress.txt
- 用描述性的訊息做 git commit，讓下一個代理實例能快速理解已完成的工作

上下文重置 vs 壓縮

文章指出了一個關鍵見解：Compaction（壓縮）雖然讓代理可以繼續工作，但不能解決「上下文焦慮（context anxiety）」問題——代理感知到上下文快滿了，會開始做出品質下降的決策，試圖快速結束任務。上下文重置（Reset）——完全開啟一個新的對話——反而更有效，因為新的代理實例有完全乾淨的上下文，沒有「快用完了要趕快結束」的壓力。代價是需要精心設計的「交接文件（Handoff Artifact）」。

人類工程師交接班的啟發

這個架構直接借鑒了人類軟體工程師的工作方式：輪班制度。每個工程師下班前會寫交接文件，說明今天做了什麼、遇到什麼問題、下一步要做什麼。下一位工程師看了交接文件，就能快速上手繼續工作。claude-progress.txt + git history 就是這個交接文件的 AI 版本。

延伸思考

長時任務框架代表了 AI 工程的一個重要認知轉變：代理不再是「一個對話中的一次性助手」，而是可以在多個會話中持續工作的「自主工程師」。這要求工程師不只思考模型能力，還要思考工作環境設計、狀態持久化、任務分解策略——這些都是傳統軟體工程的核心問題，現在需要在 AI 代理的場景下重新解答。

10

EVALUATION · TESTING · AGENT QUALITY ASSURANCE

Demystifying Evals for AI Agents

January 9, 2026 · anthropic.com/engineering/demystifying-evals-for-ai-agents

背景知識：為什麼評估（Evals）很重要？

在傳統軟體開發中，單元測試和集成測試讓你對系統行為有確定性的保障。但 AI 代理是不確定性的（stochastic）——同樣的輸入可能產生不同的輸出，代理每次執行的路徑也可能不同。沒有評估體系，開發者面對的是「飛盲狀態」：

- 不知道改動是否讓代理變好還是變差
- 使用者反映代理「感覺變差了」，但無法找到確切原因
- 無法在上線前自動測試數百個場景
- 修一個 bug 可能悄悄破壞了另一個功能

代理評估的特殊挑戰

傳統軟體測試假設「給定輸入 X，系統應該走路徑 Y 得到輸出 Z」。代理測試必須面對更複雜的情況：

多步驟互動	代理需要多輪工具呼叫才能完成任務， 評估必須考慮整個對話軌跡，而不只是最終答案
工具呼叫	工具呼叫有副作用（如修改資料庫、發送郵件）， 測試環境必須能安全地模擬這些操作
不確定性路徑	同樣的任務，代理可能走完全不同的路徑， 但兩條路徑都可能是「正確的」
評估指標設計	不像分類任務有明確的正確答案， 代理任務的「成功」往往需要主觀判斷

代理評估的組成元素

文章定義了評估系統的基本構件：

- 代理框架（Agent Harness）：讓模型作為代理運行的系統，包含工具呼叫、狀態管理等
- 評估套件（Evaluation Suite）：針對特定能力或行為的測試任務集合
- 評估指標（Metrics）：衡量成功與否的標準，可以是確定性的（測試通過/失敗）或 LLM 評判的

不同類型代理的評估策略

程式碼代理（Code Agents）

最容易評估，因為有客觀的成功標準：代碼能否通過測試。SWE-bench 和 Terminal-Bench 是兩個常用的基準。Claude Code 的評估演進：從「個人員工和外部使用者的回饋」→ 針對特定行為（如簡潔性、檔案編輯）的窄範圍評估 → 複雜行為（如過度工程化）的評估。

研究代理（Research Agents）

需要組合多種評估器：事實核查（LLM 驗證陳述是否有來源支持）、覆蓋率（是否探索了問題的所有重要面向）、來源品質（是否優先引用權威來源而非 SEO 農場）、格式一致性。人工評估員在這裡特別重要——自動指標容易遺漏細微的品質問題。

瀏覽器/電腦使用代理（Browser/Computer Use Agents）

需要在真實或沙箱環境中運行，使用 URL 和頁面狀態檢查、後端資料庫驗證。WebArena 和 OSWorld 是常用基準。Anthropic 的 Claude for Chrome 評估了代理在不同情境下選擇正確工具的能力（DOM 操作 vs 截圖），這顯著改善了任務完成率。

pass@k 指標的運用

由於代理行為有隨機性，「代理在一次嘗試中是否成功」不是好的評估指標。pass@k 衡量的是：代理在 k 次嘗試中至少有一次成功的機率。k 越大，pass@k 越高，但成本也越高。工程決策是選擇在什麼 k 值下評估，以平衡準確性和評估成本。

「評估驅動開發」：如何開始？

Anthropic 觀察到很多團隊以「評估套件要夠大才有用」為由推遲建立評估體系。實際上，哪怕只有 15–20 個基於真實失敗案例的測試任務，就能在早期發現主要的退步（regression）。建議的起步方法：

① 從真實失敗案例建立測試：收集代理在生產中出錯的例子

② 先定義評估，再讓代理通過：「評估驅動開發」的核心原則

③ LLM 作為評判者：讓另一個 LLM 用結構化量表評分，適合需要主觀判斷的場景

④ 不要跳過人工測試：自動評估找不到的細微問題，人工評估員往往能夠察覺

延伸思考

評估體系是 AI 代理從「玩具原型」到「生產系統」的關鍵橋樑。沒有評估，你無法對代理的行為有任何可信的預期，也無法對改動有信心。「測試覆蓋率」的概念需要在 AI 代理領域重新定義：不是程式碼行覆蓋率，而是代理行為空間的覆蓋率——你的評估套件能否代表代理在生產中會遇到的情況？

11

MULTI-AGENT · LARGE-SCALE SYSTEMS · COMPILER

Building a C Compiler with a Team of Parallel Claudes

February 5, 2026 · anthropic.com/engineering/building-c-compiler

背景知識：C 編譯器有多複雜？

C 編譯器是軟體工程中最複雜的系統之一。它需要實作：詞法分析（Lexer）、語法分析（Parser）、語義分析（Semantic Analyzer）、中間表示（IR）生成、代碼優化、目標碼生成等多個高度複雜的模組，且每個模組都需要對 C 語言標準有深入理解。Rust 是一種強調記憶體安全的系統程式設計語言，以 Rust 實作 C 編譯器意味著需要同時掌握兩種複雜語言的深層特性。

實驗設計

Anthropic 安全研究員 Nicholas Carlini 設計了這個實驗：使用 16 個並行的 Claude 代理實例，在共享的 Git 倉庫上協作構建一個 Rust 版本的 C 編譯器，且在整個過程中不需要（或極少需要）人工干預。

多代理協調的核心挑戰

和單代理任務不同，多代理共同修改同一個程式碼庫引入了全新的工程挑戰：

- 程式碼衝突（Code Conflicts）：多個代理同時修改相同的模組時，如何合併衝突的變更？
- 介面一致性（Interface Consistency）：代理 A 定義的函式 API，代理 B 必須以相同方式呼叫
- 工作分配（Task Allocation）：如何讓 16 個代理各自處理不同的模組，避免重複工作？
- 整體品質（Overall Quality）：沒有人類工程師全程監督，如何確保代碼品質和一致性？

關鍵結果

16	100,000+	~	完 數
並行 Claude 代理實例數	最終生成的 Rust 程式碼行數	人工干預次數（極少）	C 編譯器功能覆蓋程度

此實驗的意義

這個實驗的重要性不在於「建了一個 C 編譯器」，而在於它展示了幾個關鍵命題：

① AI 代理可以可靠地協作完成真實世界的大型複雜工程任務

② 多代理系統不需要持續的人工監督也能產出高品質輸出

③ 現有的工程工具（Git、測試框架）可以作為多代理協調的基礎設施

④ 代理的「智能」足以理解並遵循複雜的語言規範（C 語言標準）

對 AI 軟體工程未來的啟示

這個實驗是「AI 軟體工程師團隊」概念的第一個大規模驗證。未來的軟體開發可能不再是「一個人類工程師使用 AI 工具輔助」，而是「一個小型人類團隊監督大量 AI 代理並行工作」。這對軟體工程師的職業發展有深遠影響：核心技能將從「寫代碼」轉向「架構設計、任務分解、AI 代理監督和品質管控」。

延伸思考

這個實驗也提出了若干開放問題：多代理系統在什麼類型的任務上效果最好？協作模式（Collaboration Pattern）如何影響最終品質？如何衡量和確保多代理生成的代碼的長期可維護性？這些問題的答案將塑造未來 AI 輔助軟體工程的最佳實踐。

12

MCP · DESKTOP EXTENSIONS · ECOSYSTEM

Desktop Extensions: One-Click MCP Server Installation for Claude Desktop

June 26, 2025 · anthropic.com/engineering/desktop-extensions

MCP 生態系的安裝痛點

MCP 協議的推出讓 Claude 能夠連接各式各樣的外部工具，但對非技術使用者來說，安裝和配置 MCP 伺服器需要手動編輯 JSON 配置檔案、設置環境變數、安裝依賴套件——門檻相當高。這阻礙了 MCP 生態系的普及，尤其是對那些希望使用 Claude 但不具備命令列操作能力的使用者。

Desktop Extensions (.dxt) 格式

.dxt 是一種新的封裝格式，把 MCP 伺服器及其所有依賴打包成單一可安裝的檔案，就像手機 App 一樣——點一下就安裝，不需要任何命令列操作或手動配置。

傳統安裝方式	手動編輯 config JSON → 設置環境變數 → 安裝 npm/pip 依賴 → 重啟 Claude
.dxt 安裝方式	下載 .dxt 檔案 → 雙擊安裝 → 完成

對 MCP 生態的意義

Desktop Extensions 解決了 MCP 生態「最後一哩路」的問題。技術上出色的 MCP 伺服器，如果安裝過程繁瑣，普通使用者就不會去用，生態也無從發展。一鍵安裝大幅降低了使用者嘗試新工具的摩擦力，有助於形成更活躍的 MCP 工具市場。

安全性考量

降低安裝門檻的同時，安全性不能犧牲。Anthropic 設計了沙箱隔離機制，讓每個 MCP 伺服器在受限的環境中運行，無法存取不應存取的系統資源。同時提供清晰的權限說明，讓使用者在安裝前了解這個工具需要哪些存取權限。

對開發者的影響

對 MCP 工具開發者來說，.dxt 格式提供了一個標準化的發布管道。開發者可以專注於工具功能本身，而不需要為不同平台、不同用戶環境的安裝問題煩惱。這降低了 MCP 工具生態的開發門檻，有助於催生更多創新的 Claude 擴充功能。

延伸思考

Desktop Extensions 的推出讓我們看到 Anthropic 在 MCP 生態策略上的清晰思路：先建立開放的協議標準（MCP）→ 建立參考實作（Claude Desktop）→ 建立發布和分發機制（Desktop Extensions）。這條路和 Apple 的 App Store 策略有異曲同工之妙。差異在於 MCP 是開放的——任何 AI 助手都可以採用，這有助於建立更廣泛的生態互惠。

總結：Anthropic Engineering Blog 的核心洞見

回顧這 12 篇文章，可以提煉出幾個貫穿始終的核心主題：

主題一：從複雜走向簡單

Anthropic 反覆強調「最成功的實作使用簡單、可組合的模式」。這不是說複雜系統沒有價值，而是說複雜度必須有充分的理由——只在能明確證明帶來改善時才增加。這個原則對抗的是工程師的「過度設計」本能。

主題二：上下文是核心稀缺資源

從 Contextual Retrieval 到 Context Engineering，從 Code Execution with MCP 到 Long-Running Harnesses，每篇文章都在從不同角度處理同一個問題：如何在有限的上下文窗口中放入最有價值的資訊，讓模型做出最好的決策。這是 AI 工程師的核心技能。

主題三：評估先行

沒有評估，你不知道自己是否在改善系統。Anthropic 的 Claude Code 就是先從「直覺和回饋」驅動開發，後來逐漸建立評估體系——這讓他們有了可靠的信號來引導改進方向。對所有 AI 代理開發者而言，建立評估體系的優先級應該遠高於添加新功能。

主題四：向人類工程師學習

Long-Running Harnesses 的靈感來自工程師的「輪班交接」；Multi-Agent Research System 模仿的是「研究團隊分工」；Building Effective Agents 提醒我們工具設計的重要性——這些都是人類工程師幾十年積累的智慧。在 AI 代理的設計上，往往最好的答案已經存在於人類的工作實踐中，只需要翻譯成 AI 的語言。

主題五：生態建設與標準化

MCP 的推出和 Desktop Extensions 的設計，展示了 Anthropic 不只在建造更好的模型，也在為整個 AI 工具生態系建立基礎設施。開放標準、降低門檻、讓更多人能夠建造和使用 AI 工具——這是一個更宏大的願景。

對中文讀者的建議

Anthropic 的 Engineering Blog 是學習前沿 AI 工程實踐的最佳資源之一，所有文章均免費公開在 anthropic.com/engineering。推薦訂閱 Anthropic 的官方 Newsletter，第一時間獲取新文章。對於希望深入實踐的讀者，Anthropic 的 GitHub (github.com/anthropics) 提供了豐富的 Cookbook 和示例代碼，包括 Contextual Retrieval 的完整實作。

整理製作：2026 年 3 月 | 基於 anthropic.com/engineering 公開文章

本文件僅供學習用途，文章版權歸 Anthropic PBC 所有。